# Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINNERING**

**By**

**G.V.S Hasini-2420030735**

**G.Hemalatha-2420030703**

**K.Akshitha-2420030657**

Under the Esteemed Guidance of

**Srilakshmi Potluri**




**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**




# DECLARATION

The Mini Project Report entitled **Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance** is a record of Bonafide work of **G.V.S Hasini-2420030735,G.Hemalatha-2420030703,K.Akshitha-2420030657**. submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**G.V.S Hasini–2420030735**

**G.Hemalatha-2420030703**

**K.Akshitha – 2420030657**

## K L (Deemed to be) University

## DEPARTMENT OF COMPUTER SCIENCE ENGINEERING




# CERTIFICATE

This is certify that the mini project based report entitled **Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance** is a bonafide work done and submitted by **G.V.S Hasini-2420030735,G.Hemalatha-2420030703,K.Akshitha-2420030657 .** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

**Signature of the Course Co-ordinator** **Signature of HOD**

# ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Srilakshmi Potlur**i, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)/Dr. Kaja Shareef Sir (CSIT)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

**G.V.S Hasini–2420030735**

**G.Hemalatha-2420030703**

**K.Akshitha – 2420030657**

# TABLE OF CONTENTS


| S.No. | Contents | Page no. |
|---|---|---|
| 1. | Abstract | |
| 2. | Introduction | |
| 3. | Literature survey | |
| 4. | Problem Statement and Objectives | |
| 5. | Hardware and Software requirements | |
| 6. | Implementation | |
| 7. | Results & Discussion | |
| 8. | Conclusion & Future Scope | |
| 9. | References | |

# 1. ABSTRACT

The Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance is an AI-powered system designed to provide students, faculty, and university staff with fast, accurate, and reliable access to academic information. In many universities, students often spend significant time searching through websites, notices, PDFs, or contacting administrative offices to find details about course syllabi, examination schedules, attendance policies, fee structures, library services, placement opportunities, and academic regulations. Traditional chatbots are generally rule-based and limited to predefined responses, making them incapable of handling complex or newly updated queries. This creates delays, increases the workload on university staff, and may result in inconsistent or outdated information being shared.

The proposed system overcomes these limitations by using the Retrieval-Augmented Generation (RAG) framework, which combines intelligent document retrieval with the language generation capabilities of a Large Language Model (LLM). When a user submits a question, the chatbot first searches a repository of official university documents to retrieve the most relevant information. It then uses the retrieved content to generate a clear, natural, and context-aware response. This approach ensures that the answers are based on authentic and up-to-date university resources, significantly reducing the chances of incorrect or fabricated information while improving the overall reliability of the system.

The RAG chatbot supports natural language interactions, allowing users to ask questions in a conversational manner without requiring specific commands or keywords. The system provides instant responses, is available 24/7, and can be easily updated by adding new university documents to its knowledge base. As a result, it enhances the accessibility of academic information, saves time for students, reduces the burden on faculty and administrative staff, and improves the overall efficiency of university support services. Furthermore, the chatbot can be extended with features such as multilingual support, voice-based interaction, and personalized recommendations, making it a scalable and intelligent solution for modern educational institutions.

**Team Members:**

G.V.S Hasini-2420030735

G.Hemalatha-2420030703

K.Akshitha-2420030657

# 2. INTRODUCTION

Universities provide a large amount of academic information, including course details, syllabi, examination schedules, attendance rules, academic regulations, notices, and library services. Students often find it difficult and time-consuming to search through different university websites, portals, and PDF documents to find the information they need. They may also need to contact faculty members or administrative staff for answers to common academic questions. This creates delays for students and increases the workload of university staff.

To overcome these challenges, this project proposes a Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance. The main aim of this system is to provide students, faculty, and university staff with quick and reliable answers to academic queries through a simple conversational interface. Users can ask questions in natural language without needing to search manually through multiple documents or use specific commands.

The proposed system uses Retrieval-Augmented Generation (RAG), which combines information retrieval with the text-generation capabilities of a Large Language Model (LLM). When a user submits a question, the system searches a knowledge base containing university documents and retrieves the most relevant information. The retrieved content is then used by the language model to generate a clear and context-aware response. By grounding responses in university-specific information, the system aims to reduce incorrect answers and provide more relevant academic assistance.

The chatbot is designed to make academic information more accessible, save students' time, and reduce the workload of administrative staff by answering frequently asked questions. The knowledge base can also be updated by adding new university documents, helping the system provide information based on available resources. The proposed approach offers a convenient way to access academic information and can be extended in the future with features such as multilingual support, voice interaction, and personalized recommendations.

The remainder of this paper presents the related literature and limitations of existing systems, the proposed methodology and system architecture, the implementation approach, and the conclusion with possible future enhancements.

# 3. LITERATURE SURVEY

1. APAssist – University Knowledge Base Chatbot
APAssist uses Retrieval-Augmented Generation (RAG) to answer university-related questions. It retrieves relevant information from a knowledge base. It helps students access academic information quickly.
2. University Student Services Chatbot
This chatbot uses NLP and RAG to answer questions about university services. It provides information related to student activities and university resources. It helps improve communication between students and university staff.
3. Educational Virtual Assistant
This system uses large language models and RAG to answer educational queries. It supports questions about admissions, courses, and campus facilities. It makes academic information easier for students to access.
4. AI-Driven Academic Assistant
This system uses AI to provide academic and administrative information to students. It retrieves relevant information before generating answers. It helps reduce the effort required to obtain university services information.
5. University Resources Chatbot
This chatbot helps users find information about university resources. It uses relevant information to generate answers to user questions. It reduces the need to search multiple websites manually.
6. Deep Learning University Chatbot
This system combines deep learning techniques with information retrieval. It answers questions related to university information using available knowledge sources. It aims to provide relevant responses to students.
7. Multilingual Academic Chatbot
This chatbot uses RAG to answer questions about academic regulations and examination policies. It supports multilingual communication and voice-based interaction. It improves accessibility for students with different language preferences.
8. RAG Chatbots for Education
This study examines the use of RAG chatbots in educational applications. It explains how retrieving relevant documents can improve answer quality. It also highlights challenges such as incorrect retrieval and inaccurate responses.

# 4. Problem Statement and Objectives

**Problem Statement:**

Students often face difficulties finding accurate and updated university academic information because it is spread across multiple websites, notices, portals, and PDF documents. This makes the process time-consuming and inconvenient.

**Problem Description:**

Students need to search through different sources to find information about course details, syllabi, examination schedules, attendance rules, and academic regulations. Traditional chatbots may provide limited or outdated answers, creating a need for an intelligent chatbot that retrieves relevant information from university documents and provides clear, reliable responses.

**Objectives:**

- To develop a RAG-based chatbot for university academic assistance.

- To provide quick and accurate answers to students' academic queries.

- To retrieve relevant information from university documents before generating responses.

- To reduce manual searching and the workload of university administrative staff.

# 5.HARDWARE AND SOFTWARE REQUIREMENTS

**Hardware Requirements:**
- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB HDD/SSD

**Software Requirements:**
- Operating System: Windows 10/11
- Programming Language: Python
- Frontend: HTML, CSS, JavaScript
- Libraries and Tools: Streamlit, LangChain, Hugging Face Transformers, FAISS/ChromaDB

# 6.IMPLEMENTATION

```
from pathlib import Path

from dotenv import load_dotenv
from pypdf import PdfReader
from docx import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document as LCDocument

load_dotenv(override=True)

BASE_DIR = Path(__file__).resolve().parent
DOCS_DIR = BASE_DIR / "documents"
VECTOR_DIR = BASE_DIR / "vectorstore"

EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"


def read_file(path):
    suffix = path.suffix.lower()

    if suffix == ".pdf":
        reader = PdfReader(str(path))
        return "\n".join(page.extract_text() or "" for page in reader.pages).strip()

    if suffix == ".txt":
        return path.read_text(encoding="utf-8", errors="ignore").strip()

    if suffix == ".docx":
        doc = Document(str(path))
        return "\n".join(p.text for p in doc.paragraphs).strip()

    return ""


def get_embeddings():
    return HuggingFaceEmbeddings(
        model_name=EMBEDDING_MODEL,
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True}
    )


def build_index():
    DOCS_DIR.mkdir(parents=True, exist_ok=True)
    documents = []

    for path in sorted(DOCS_DIR.iterdir()):
        if path.is_file() and path.suffix.lower() in {".pdf", ".txt", ".docx"}:
            text = read_file(path)

            if text:
                documents.append(
                    LCDocument(
                        page_content=text,
                        metadata={"source": path.name}
                    )
                )
```

```
    if not documents:
        raise ValueError("No readable documents found in the documents folder.")

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800,
        chunk_overlap=120
    )

    chunks = splitter.split_documents(documents)
    db = FAISS.from_documents(chunks, get_embeddings())

    VECTOR_DIR.mkdir(parents=True, exist_ok=True)
    db.save_local(str(VECTOR_DIR))

    return len(chunks)


def index_exists():
    return (
        (VECTOR_DIR / "index.faiss").exists()
        and (VECTOR_DIR / "index.pkl").exists()
    )


def get_database():
    if not index_exists():
        build_index()

    return FAISS.load_local(
        str(VECTOR_DIR),
        get_embeddings(),
        allow_dangerous_deserialization=True
    )


def ask_rag(question):
    question = question.strip()
    q = question.lower()

    if not question:
        return {"answer": "Please enter a question.", "sources": []}

    db = get_database()
    results = db.similarity_search_with_score(question, k=4)

    useful = [(doc, score) for doc, score in results if score < 1.30]

    if not useful:
        return {
            "answer": "I couldn't find this information in the university documents.",
            "sources": []
        }

    text = "\n".join(doc.page_content for doc, score in useful).lower()
    sources = list(dict.fromkeys(
        doc.metadata.get("source", "University document")
        for doc, score in useful
    ))

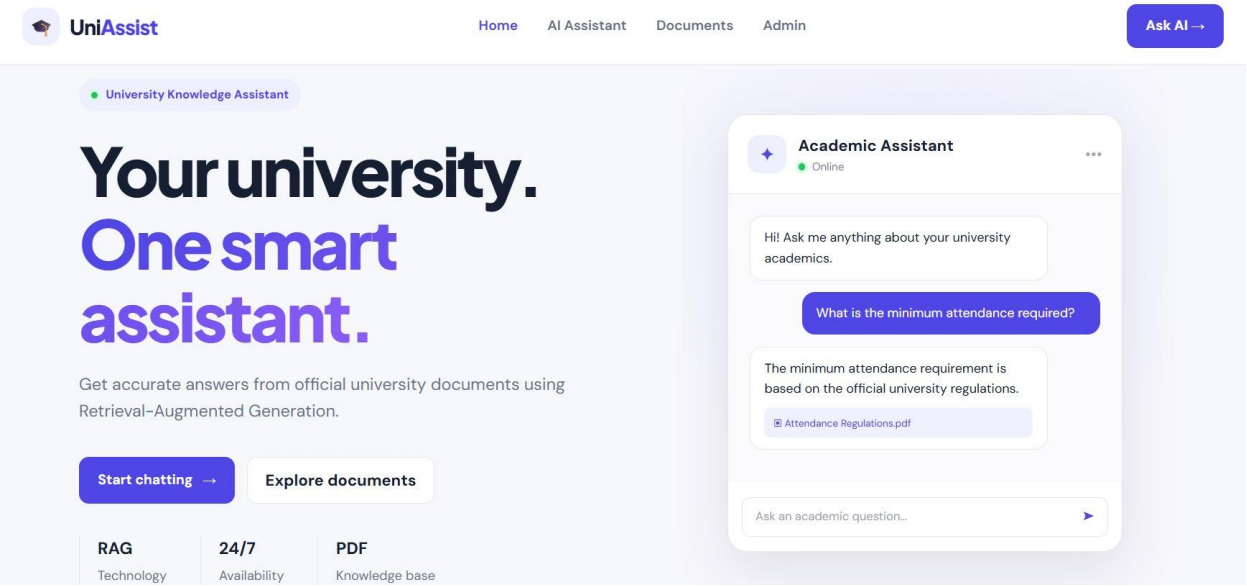
    if "attendance" in q or "eligible" in q:
        if "75%" in text or "75 percent" in text:
```

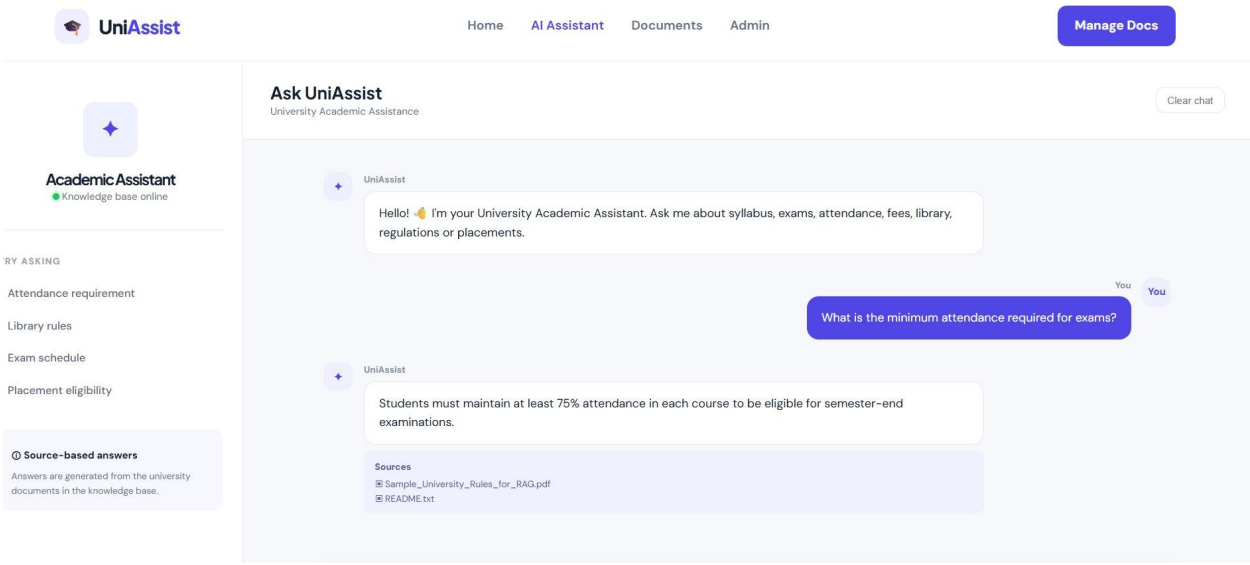
```
        answer = (
            "Students must maintain at least 75% attendance in each "
            "course to be eligible for semester-end examinations."
        )
    else:
        answer = "I couldn't find the required attendance percentage in the retrieved documents."

elif "exam" in q or "examination" in q:
    if "15 minutes" in text:
        answer = (
            "Students must carry their valid university ID card, arrive "
            "at least 15 minutes before the exam, and avoid mobile phones, "
            "smart watches, unauthorized notes, and other prohibited materials."
        )
    else:
        answer = "\n\n".join(doc.page_content.strip() for doc, score in useful)

elif "plagiarism" in q or "academic integrity" in q:
    answer = (
        "Students must submit their own
```

# 7.RESULTS & DISCUSSION



# 8.CONCLUSION & FUTURE SCOPE

**Conclusion**

The Retrieval-Augmented Generation (RAG) Chatbot for University Academic Assistance is proposed to make academic information easily accessible to students, faculty, and university staff. The system combines document retrieval with the text-generation capabilities of a Large Language Model (LLM) to provide relevant and context-aware answers to user queries. It helps students find information about course details, syllabi, examination schedules, attendance rules, academic regulations, and university notices without searching through multiple sources manually.

The proposed system aims to save time, improve access to academic information, and reduce the workload of administrative staff by answering frequently asked questions. The knowledge base can be updated by adding new university documents, allowing the chatbot to use the latest available information. Overall, the project provides a convenient approach to university academic assistance, with the potential to improve the accessibility and reliability of information services.

**2. Future Scope**

- Add more university documents, academic notices, syllabi, and examination schedules to the knowledge base.
- Improve document retrieval so that the chatbot can identify the most relevant information for each question.
- Provide source references with answers so that users can verify the information.
- Introduce multilingual support to help students ask questions in different languages.
- Add voice input and voice-based responses for easier interaction.
- Improve response quality by evaluating the accuracy, relevance, and reliability of generated answers.
- Integrate the chatbot with university websites and student portals for easier access to academic information.
- Introduce personalized academic assistance based on students' courses and academic requirements.
- Extend the system to support multiple departments and universities by maintaining separate knowledge bases for each institution.

These improvements can make the chatbot more flexible, user-friendly, and useful for providing academic assistance in modern educational institutions.

# 9.REFERENCES

1. APAssist — University Knowledge Base RAG Chatbot
https://kamilhassan.com/project/apassist

2. University Chatbot for Student Services — IHEC Hackathon
https://ahmedaziz-portfolio.vercel.app/projects/university-chatbot-student-services

3. Large Language Model Based Educational Virtual Assistant Using RAG Framework
https://doi.org/10.1016/j.procs.2025.01.051

4. Enhancing Student Academic Services Through AI-Driven Virtual Assistants Using RAG
https://doi.org/10.21831/jitp.v13i1.83632

5. From Questions to Insightful Answers: Building an Informed Chatbot for University Resources
https://arxiv.org/abs/2405.08120

6. Development and Evaluation of a University Chatbot Using Deep Learning: A RAG-Based Approach
https://doi.org/10.1007/978-3-031-88045-2_7

7. An On-Premise Multilingual Academic Chatbot Using RAG and Context-Aware Memory
https://ieeexplore.ieee.org/abstract/document/11656515/

8. Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications
https://www.mdpi.com/2076-3417/15/8/4234

9. Retrieval-Augmented Generation for Educational Application: A Systematic Survey
https://doi.org/10.1016/j.caeai.2025.100417

The ninth reference is an additional survey that reviews RAG techniques and their applications in education.